

4/9/26 DAY 12:

STL Containers

STL (Standard Template Library) provides container classes that store collections of data. Each container has different characteristics suitable for different use cases.

vector

- A dynamic array that can grow or shrink in size at runtime.
- Elements are stored contiguously in memory for fast random access.
- Use when you need dynamic arrays with frequent access by index.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Creating and initializing vectors
    vector<int> v1;                // empty vector
    vector<int> v2 = {10, 20, 30, 40}; // initialized with values
    vector<int> v3(5, 100);        // 5 elements, all initialized to 100

    // Adding elements
    v1.push_back(10);
    v1.push_back(20);
    v1.emplace_back(30);           // similar to push_back

    // Accessing elements
    cout << v1[0] << endl;         // using index
    cout << v1.at(1) << endl;       // bounds checking
    cout << v1.front() << endl;     // first element
    cout << v1.back() << endl;     // last element

    // Size and capacity
    cout << v1.size() << endl;      // number of elements
    cout << v1.capacity() << endl; // allocated space

    // Removing elements
    v1.pop_back();                 // remove last element
    v1.erase(v1.begin());          // remove first element

    // Iteration
    for (int val : v1) {
        cout << val << " ";
    }

    // Clear vector
    v1.clear();
```

```
    return 0;
}
```

stack

- LIFO (Last-In-First-Out) data structure where elements are added and removed from the top.
- Use when you need a stack-like behavior (undo-redo, function call stack).

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;

    // Push elements
    st.push(10);
    st.push(20);
    st.push(30);

    // Access top element
    cout << st.top() << endl;    // prints 30

    // Pop element
    st.pop();                    // removes 30

    // Check if empty
    while (!st.empty()) {
        cout << st.top() << " ";
        st.pop();
    }

    return 0;
}
```

queue

- FIFO (First-In-First-Out) data structure where elements are added at the back and removed from the front.
- Use when you need queue-like behavior (task scheduling, breadth-first search).

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    // Enqueue elements
```

```

    q.push(10);
    q.push(20);
    q.push(30);

    // Access front element
    cout << q.front() << endl;    // prints 10

    // Dequeue element
    q.pop();                      // removes 10

    // Check if empty
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }

    return 0;
}

```

map

- Key-value pair container that maintains sorted order based on keys.
- Each key appears only once (associative container).
- Use when you need to associate values with unique keys.

```

#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, string> m;

    // Inserting elements
    m[101] = "Shil";
    m[102] = "Priya";
    m[103] = "Ravi";
    m.insert({104, "Aniket"});

    // Accessing elements
    cout << m[101] << endl;    // prints "Shil"

    // Checking if key exists
    if (m.find(102) != m.end()) {
        cout << "Key found!" << endl;
    }

    // Size
    cout << m.size() << endl;

    // Iteration
    for (auto& pair : m) {

```

```
        cout << pair.first << " -> " << pair.second << endl;
    }

    // Removing elements
    m.erase(101);

    return 0;
}
```

Iterators — Introduction

- Iterators are objects that point to elements in containers and provide a way to traverse them.
- Similar to pointers but work with all STL containers uniformly.
- Types: input, output, forward, bidirectional, and random access iterators.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40, 50};

    // Using iterators
    vector<int>::iterator it = v.begin(); // points to first element
    cout << *it << endl;                 // dereference to access value

    // Increment iterator
    ++it;
    cout << *it << endl;                 // prints 20

    // Iterate using iterator
    for (auto it = v.begin(); it != v.end(); ++it) {
        cout << *it << " ";
    }

    // Reverse iterator
    for (auto rit = v.rbegin(); rit != v.rend(); ++rit) {
        cout << *rit << " ";
    }

    return 0;
}
```

Algorithms — Basic Usage

STL provides ready-to-use algorithms that work with containers and iterators.

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30, 40, 30, 50};

    // find - search for an element
    auto it = find(v.begin(), v.end(), 30);
    if (it != v.end()) {
        cout << "Found at position: " << (it - v.begin()) << endl;
    }

    // count - count occurrences
    int cnt = count(v.begin(), v.end(), 30);
    cout << "Count: " << cnt << endl;

    // sort - sort elements
    sort(v.begin(), v.end());

    // reverse - reverse elements
    reverse(v.begin(), v.end());

    // max_element and min_element
    int max_val = *max_element(v.begin(), v.end());
    int min_val = *min_element(v.begin(), v.end());

    // for_each - apply function to each element
    for_each(v.begin(), v.end(), [](int x) {
        cout << x << " ";
    });

    return 0;
}
```

Introduction to RTTI (Run-Time Type Information)

RTTI allows programs to obtain information about objects at runtime, enabling dynamic type checking and casting.

typeid

- Returns type information of an expression or type.
- Available in `<typeinfo>` header.
- Useful for debugging and type-aware programming.

```
#include <iostream>
#include <typeinfo>
using namespace std;

class Animal {};
```

```

class Dog : public Animal {};

int main() {
    Animal a;
    Dog d;
    Animal* ptr = &d;

    // Get type information
    cout << typeid(a).name() << endl;    // prints "class Animal"
    cout << typeid(d).name() << endl;    // prints "class Dog"
    cout << typeid(*ptr).name() << endl;  // prints "class Animal" (static type)

    // Compare types
    if (typeid(a) == typeid(d)) {
        cout << "Same type" << endl;
    } else {
        cout << "Different types" << endl;
    }

    return 0;
}

```

static_cast

- Compile-time cast that performs type conversions when the compiler can verify safety.
- Used for converting related types (base to derived, numeric conversions).

```

#include <iostream>
using namespace std;

int main() {
    double d = 3.14;
    int i = static_cast<int>(d);  // convert double to int
    cout << i << endl;           // prints 3

    int* ptr = &i;
    void* v_ptr = static_cast<void*>(ptr);  // convert int* to void*

    return 0;
}

```

dynamic_cast

- Runtime cast used for polymorphic types (requires virtual functions).
- Safely converts pointers and references in a class hierarchy.
- Returns nullptr for pointers if cast fails; throws bad_cast for references.

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual ~Animal() {}
};

class Dog : public Animal {};
class Cat : public Animal {};

int main() {
    Animal* animal = new Dog();

    // Safe cast
    Dog* dog = dynamic_cast<Dog*>(animal);
    if (dog != nullptr) {
        cout << "Successfully cast to Dog!" << endl;
    }

    // Unsafe cast
    Cat* cat = dynamic_cast<Cat*>(animal);
    if (cat == nullptr) {
        cout << "Cannot cast to Cat!" << endl;
    }

    delete animal;
    return 0;
}
```

const_cast

- Removes const or volatile qualifiers from a pointer or reference.
- Used when you need to pass const data to a function that doesn't take const parameters.

```
#include <iostream>
using namespace std;

int main() {
    const int a = 10;
    const int* ptr = &a;

    // Remove const qualifier
    int* non_const_ptr = const_cast<int*>(ptr);
    *non_const_ptr = 20; // may result in undefined behavior

    const string s = "Hello";
    string& ref = const_cast<string&>(s);
}
```

```
    return 0;
}
```

reinterpret_cast

- Converts between unrelated pointer or reference types.
- Performs low-level reinterpretation of bits.
- Dangerous and should be used only when necessary.

```
#include <iostream>
using namespace std;

int main() {
    int a = 65;
    char* ptr = reinterpret_cast<char*>(&a);

    // Useful for binary file I/O
    struct Data {
        int id;
        double value;
    };

    Data d = {1, 3.14};
    char* binary = reinterpret_cast<char*>(&d); // for file writing

    return 0;
}
```

Exception Handling Introduction

Exception handling allows programs to detect and respond to errors gracefully without terminating abnormally.

try-catch Blocks

- **try** block: contains code that might throw an exception.
- **catch** block: handles the exception if thrown.

```
#include <iostream>
using namespace std;

int main() {
    try {
        int a = 10, b = 0;
        if (b == 0) {
            throw runtime_error("Division by zero!");
        }
    }
}
```



```
        int c = a / b;
    }
    catch (const runtime_error& e) {
        cout << "Caught exception: " << e.what() << endl;
    }

    return 0;
}
```

Multiple Catch Blocks

- Handle different exception types with separate catch blocks.
- Executed in order from top to bottom.

```
#include <iostream>
using namespace std;

int main() {
    try {
        throw 42; // throwing an int
    }
    catch (const string& e) {
        cout << "String exception: " << e << endl;
    }
    catch (const int& e) {
        cout << "Integer exception: " << e << endl;
    }
    catch (...) { // catch-all handler
        cout << "Unknown exception" << endl;
    }

    return 0;
}
```

Nested try-catch

- try-catch blocks can be nested within each other.

```
#include <iostream>
using namespace std;

int main() {
    try {
        try {
            throw runtime_error("Inner exception");
        }
        catch (const runtime_error& e) {
            cout << "Inner catch: " << e.what() << endl;
            throw; // rethrow
        }
    }
}
```

```

    }
}
catch (const runtime_error& e) {
    cout << "Outer catch: " << e.what() << endl;
}

return 0;
}

```

Rethrowing Exceptions

- Use `throw;` to rethrow the current exception to an outer catch block.

```

#include <iostream>
using namespace std;

void process() {
    try {
        throw runtime_error("Processing error");
    }
    catch (const exception& e) {
        cout << "Processing: " << e.what() << endl;
        throw; // rethrow for caller to handle
    }
}

int main() {
    try {
        process();
    }
    catch (const exception& e) {
        cout << "Main: " << e.what() << endl;
    }

    return 0;
}

```

Custom Exception Class

- Derive from `std::exception` to create custom exceptions.

```

#include <iostream>
#include <exception>
using namespace std;

class AgeException : public exception {
private:
    string message;
public:

```

```

    AgeException(string msg) : message(msg) {}

    const char* what() const noexcept override {
        return message.c_str();
    }
};

int main() {
    try {
        int age = -5;
        if (age < 0) {
            throw AgeException("Age cannot be negative!");
        }
    }
    catch (const AgeException& e) {
        cout << "Custom exception: " << e.what() << endl;
    }

    return 0;
}

```

Console I/O Operations

C++ provides streams for input and output operations with formatting capabilities.

C++ Streams

- Streams are sequences of characters flowing between programs and I/O devices.
- Input streams (istream), Output streams (ostream), Bidirectional streams (iostream).

C++ Stream Classes

- `istream` - input stream base class
- `ostream` - output stream base class
- `iostream` - bidirectional stream (inherits from istream and ostream)
- `ifstream` - file input stream
- `ofstream` - file output stream
- `fstream` - file bidirectional stream

cin and cout

- `cin` - standard input stream for reading from keyboard
- `cout` - standard output stream for writing to screen

```

#include <iostream>
using namespace std;

int main() {
    int age;

```

```
string name;

cout << "Enter your name: ";
cin >> name;

cout << "Enter your age: ";
cin >> age;

cout << "Name: " << name << ", Age: " << age << endl;

return 0;
}
```

cerr and clog

- `cerr` - standard error stream (unbuffered)
- `clog` - standard error stream (buffered)

```
#include <iostream>
using namespace std;

int main() {
    cerr << "This is an error message" << endl;
    clog << "This is a log message" << endl;

    return 0;
}
```

Formatted I/O

Manipulators control how data is formatted during input/output.

setw

- Sets the field width for the next output operation.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << setw(10) << "Name" << setw(10) << "Age" << endl;
    cout << setw(10) << "Shil" << setw(10) << 20 << endl;

    return 0;
}
```

setprecision and fixed

- `setprecision` - sets decimal precision
- `fixed` - displays fixed-point notation

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double pi = 3.14159265;

    cout << pi << endl;                // default precision
    cout << setprecision(2) << pi << endl;    // 3.1
    cout << fixed << setprecision(2) << pi << endl; // 3.14

    return 0;
}
```

setfill

- Sets the fill character for padding.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << setfill('*') << setw(20) << "Name" << endl;
    cout << setfill('-') << setw(20) << 42 << endl;

    return 0;
}
```

Unformatted I/O

Reading raw characters without format interpretation.

getline

- Reads an entire line including spaces.

```
#include <iostream>
using namespace std;

int main() {
    string line;
    cout << "Enter a line: ";
}
```

```
getline(cin, line);
cout << "You entered: " << line << endl;

return 0;
}
```

cin.get

- Reads a single character from input.

```
#include <iostream>
using namespace std;

int main() {
    char ch;
    cout << "Enter a character: ";
    cin.get(ch);
    cout << "You entered: " << ch << endl;

    return 0;
}
```

File Handling in C++

File handling allows reading and writing data to/from files on the disk.

ifstream

- Input file stream for reading data from files.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream infile("data.txt");

    if (!infile.is_open()) {
        cout << "Error opening file" << endl;
        return 1;
    }

    string line;
    while (getline(infile, line)) {
        cout << line << endl;
    }

    infile.close();
}
```

```
    return 0;
}
```

ofstream

- Output file stream for writing data to files.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outfile("data.txt");

    if (!outfile.is_open()) {
        cout << "Error creating file" << endl;
        return 1;
    }

    outfile << "Hello World" << endl;
    outfile << "This is a test file" << endl;

    outfile.close();

    return 0;
}
```

fstream

- Bidirectional file stream for both reading and writing.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file("data.txt", ios::in | ios::out);

    if (!file.is_open()) {
        cout << "Error opening file" << endl;
        return 1;
    }

    // Write
    file << "Hello World" << endl;

    // Move to beginning
    file.seekg(0);
}
```

```
// Read
string line;
while (getline(file, line)) {
    cout << line << endl;
}

file.close();

return 0;
}
```

File Open Modes

Modes control how files are opened:

Mode	Description
<code>ios::in</code>	Open for reading (default for ifstream)
<code>ios::out</code>	Open for writing (default for ofstream)
<code>ios::app</code>	Append to end of file
<code>ios::binary</code>	Open in binary mode
<code>ios::ate</code>	Move to end of file after opening
<code>ios::trunc</code>	Truncate (erase) existing file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Text mode - write and read
    ofstream write("data.txt", ios::out);
    write << "Hello from AC" << endl;
    write.close();

    ifstream read("data.txt", ios::in);
    string line;
    while (getline(read, line)) {
        cout << line << endl;
    }
    read.close();

    // Binary mode - write and read objects
    class Student {
    public:
        int rollno;
        char name[50];
    };
}
```



```
};

ofstream binWrite("data.dat", ios::binary);
Student s1 = {101, "Shil"};
binWrite.write(reinterpret_cast<char*>(&s1), sizeof(Student));
binWrite.close();

ifstream binRead("data.dat", ios::binary);
Student temp;
binRead.read(reinterpret_cast<char*>(&temp), sizeof(Student));
cout << "Roll: " << temp.rollno << ", Name: " << temp.name << endl;
binRead.close();

return 0;
}
```

Reading from Files

- Use `ifstream` to read data from files.
- Use `getline` for text files and `read` for binary files.
- It is important to check if the file opened successfully before reading.

Writing to Files

- Use `ofstream` to write data to files.
- Use `<<` operator for text files and `write` for binary files.
- Always close the file after operations to free resources.

Text Files

- Text files store data in human-readable format, typically using ASCII or UTF-8 encoding.
- Use text files for simple data storage and configuration files.

Binary Files

- It is stored in raw bytes, which means that the data is not formatted for human readability and may not be portable across different systems or compilers.
- Use binary files for performance and when storing non-text data like images or serialized objects.
- Binary files are more compact and faster to read/write than text files.
- The `write` and `read` methods are used for binary file operations, which require casting objects to `char*`.
- The data is written and read in the exact memory representation of the object, so care must be taken with object layout and padding.

Final OOP Reinforcement

- The principles of Object-Oriented Programming (OOP) include encapsulation, inheritance, polymorphism, and abstraction.
- Encapsulation: Bundling data and methods that operate on that data within a class.

- **Inheritance:** Creating new classes based on existing classes to promote code reuse.
- **Polymorphism:** The ability of different classes to be treated as instances of the same class through a common interface, often achieved through virtual functions.
- **Abstraction:** Hiding complex implementation details and exposing only the necessary parts of an object or system.

OOP Design Principles

According to the Grady Booch's principles, the following are key design principles in OOP:

Major Pillars of OOP

1. **Encapsulation:** Protecting the internal state of an object and requiring all interaction to be performed through an object's methods.
2. **Inheritance:** Allowing a new class to inherit properties and behavior from an existing class.
3. **Heirarchy:** Organizing classes in a tree-like structure where subclasses inherit from parent classes.
4. **Modularity:** Dividing a program into separate modules that can be developed, tested, and maintained independently.

Minor Pillars of OOP

1. **Typing:** The ability to define and use types in a programming language, ensuring that operations are performed on compatible data types.
2. **Concurrency:** The ability of a program to execute multiple parts of code simultaneously, which can improve performance and responsiveness.
3. **Persistence:** The ability of an object to maintain its state across different sessions or executions of a program, often achieved through serialization or database storage.

Modern C++ Features Covered (C++11, C++14, C++17)

1. **auto** keyword - Type Inference

- Used extensively in range-based for loops and iterators
- Reduces code verbosity and improves maintainability
- Example: `auto it = v.begin();`

2. **nullptr** - Null Pointer Constant

- Introduced in Day 3 as the proper way to represent null pointers
- Type-safe alternative to `NULL` macro
- Used throughout smart pointers and dynamic casting

3. Range-based **for** loops

- Simplified iteration over containers
- Syntax: `for (auto x : container)`
- Used in STL examples

4. Smart Pointers

- **unique_ptr**: Exclusive ownership of dynamically allocated objects
- **shared_ptr**: Shared ownership with reference counting
- **weak_ptr** : Non-owning reference to break circular dependencies
- **std::make_unique()** and **std::make_shared()**: Safe factory functions
- Provides automatic memory management and prevents memory leaks

5. Move Semantics and Rvalue References

- **Rvalue references** (&&): Bind to temporary objects
- **std::move()**: Enable efficient transfer of resources
- Improves performance by avoiding unnecessary copying
- Essential for modern C++ resource management

6. **override** keyword

- Explicitly mark virtual functions in derived classes
- Prevents accidental overload instead of override
- Enhances code clarity and catches errors at compile-time

7. Uniform Initialization with braces {}

- Consistent initialization syntax across types
 - Example: `vector<int> v = {10, 20, 30, 40};`
 - Used in STL containers
-